3 — SDOM Single Run, Outputs, and Default Plots

This module runs SDOM end-to-end on `data/sample_data/`, exports results, generates default plots, and explores key fields of `OptimizationResults`.

Learning objectives

- Build and solve an SDOM model using the public API.
- Use `N_HOURS = 7*24*2` (336 hours) for a practical training horizon in this module.
- Check solve success before post-processing.
- Export CSV outputs and generate default plots.

Prerequisites

- You completed modules 1 and 2.
- Your environment has `sdom` and a solver (HiGHS recommended for this module).

Inputs and scenario used

- Scenario folder: `data/sample_data/`
- Time horizon in this training script: `N_HOURS = 7*24*2` (336 time steps)
- Solver: HiGHS (`solver_name="highs"`)

Reference:

- Running SDOM and outputs: https://natlabrockies.github.io/SDOM/user_guide/running_and_outputs.html

Project/file structure for this module

```
training/3_sdom_single_run/
├── README_m3.md
└── run_m3.py
```

Module suffix and script naming

- Module suffix: `m3`
- Script file: `run_m3.py`

Computational burden and practical simplifications

One key SDOM advantage is that it can co-optimize both storage power capacity (MW) and storage duration (energy-to-power relationship). SDOM represents storage investment with separate power and energy CAPEX terms (`P_Capex` and `E_Capex`), which improves realism but also increases model complexity.

When many storage technologies are available (with different CAPEX structures and efficiencies), the optimization can become computationally heavy and harder to solve.

To reduce computational burden, users can:

- Fix duration for selected technologies by setting `Min_Duration = Max_Duration` in `data/sample_data/StorageData_2050.csv`.
- Reduce the number of storage technologies included in `data/sample_data/StorageData_2050.csv`.
- Use a combination of both approaches.

Note on `N_HOURS` in training scripts

`N_HOURS` in the training scripts is a didactic setting to produce lighter models and faster results. For real planning studies, runs should use a full year horizon (`8760` hours).

Step-by-step walkthrough

1. Run the script:

   ```
   python training/3_sdom_single_run/run_m3.py
   ```

2. Inspect terminal output for:
   - Solver status and termination condition.
   - Total cost and selected capacity metrics.
3. Inspect output artifacts under:

   ```
   results/training/3_sdom_single_run/
   ```

Full runnable script

See training/3_sdom_single_run/run_m3.py.

Inspect single-run API help from Python

This module introduces the core single-run workflow: `load_data -> initialize_model -> get_default_solver_config_dict -> run_solver -> export_results`, followed by `plot_results`. Use `help()` to inspect the docstrings for the exact function signatures and return values.

Run these commands from the repository root with the virtual environment active:

```
python -c "from sdom import load_data; help(load_data)"
python -c "from sdom import initialize_model; help(initialize_model)"
python -c "from sdom import get_default_solver_config_dict;
help(get_default_solver_config_dict)"
python -c "from sdom import run_solver; help(run_solver)"
python -c "from sdom import export_results; help(export_results)"
python -c "from sdom.analytic_tools import plot_results; help(plot_results)"
```

Expected output excerpts from this environment:

```
Help on function initialize_model in module sdom.optimization_main:

initialize_model(data, n_hours=8760, with_resilience_constraints=False,
model_name='SDOM_Model')
   Initialize a Pyomo SDOM optimization model (dispatcher).

   Parameters
   ----------
   data : dict
      Data dictionary as returned by :func:`sdom.io_manager.load_data`.
   n_hours : int, optional
      Number of hours to simulate (default 8760).

   Returns
   -------
   pyomo.environ.ConcreteModel
      A fully initialized Pyomo model ready for optimization, with a
      ``profiler`` attribute attached.
```

```
Help on function get_default_solver_config_dict in module sdom.optimization_main:

get_default_solver_config_dict(solver_name='cbc',
executable_path='.\\Solver\\bin\\cbc.exe', *, mip_gap=0.002, time_limit=None,
stream_solver_output=False)
   Generate a default solver configuration dictionary with standard SDOM settings.
```

```
   Parameters
   ----------
   solver_name : str, optional
      Solver to use. Supported values are 'cbc', 'highs', and 'xpress'.
   time_limit : float, optional
      Maximum solve time in seconds. Default is None (no limit).
   stream_solver_output : bool, optional
      Whether to stream solver native output live to stdout via ``tee``.

   Returns
   -------
   dict
      Configuration dictionary with solver name, executable path, solver
      options, and solve keywords.
```

```
Help on function run_solver in module sdom.optimization_main:

run_solver(model, solver_config_dict: dict, case_name: str = 'run') ->
sdom.results.OptimizationResults
   Solve the optimization model and return structured results.

   Parameters
   ----------
   model : pyomo.core.base.PyomoModel.ConcreteModel
      The Pyomo optimization model to be solved.
   solver_config_dict : dict
      Solver configuration dictionary from get_default_solver_config_dict().
   case_name : str, optional
      Case identifier for labeling results. Defaults to "run".

   Returns
   -------
   OptimizationResults
      A dataclass containing termination condition, total cost, generation,
      storage, summary, capacity, cost breakdown, and problem information.
```

```
Help on function export_results in module sdom.io_manager:

export_results(results, case: str, output_dir: str = './results_pyomo/')
   Export optimization results to CSV files.

   Output Files
   ------------
   OutputGeneration_{case}.csv
   OutputStorage_{case}.csv
   OutputSummary_{case}.csv
   OutputThermalGeneration_{case}.csv
```

```
Help on function plot_results in module sdom.analytic_tools._single:

plot_results(result: "'OptimizationResults'", output_dir: 'Optional[str]' = None,
plots_dir: 'Optional[str]' = None) -> 'None'
   Generate and save all standard plots for a single SDOM optimization run.

   Figures saved
   -------------
   - ``capacity_donut.png``
   - ``capacity_generation_donuts.png``
   - ``heatmap_{col}.png``
```

Expected outputs

- Exported CSV files for the solved case.
- Default SDOM plots generated via `plot_results`.
- Console summary of key `OptimizationResults` fields.

How to validate results

- `solver_status == "ok"`
- `termination_condition == "optimal"`
- Output directory contains CSVs and figures.

Troubleshooting

- Solver error: ensure `highspy` is installed in `.venv`.
- Non-optimal termination: reduce horizon for smoke tests, check data quality.
- Missing output files: confirm script completed after `export_results` and `plot_results`.

References

- SDOM docs home: https://natlabrockies.github.io/SDOM/index.html
- API core: https://natlabrockies.github.io/SDOM/api/core.html
- API results: https://natlabrockies.github.io/SDOM/api/results.html
- Running and outputs: https://natlabrockies.github.io/SDOM/user_guide/running_and_outputs.html